
Rethinking Benchmarking Trajectory Error Accumulation in Fluid Dynamics Datasets

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 A recent body of work has developed fully-learned fluid simulators. Methods
2 to train those include single-step training (teacher-forcing), or using unrolled
3 training. For single-step approaches, recent papers have highlighted the importance
4 of capturing high-frequency components to ensure high-fidelity autoregressive
5 rollouts over long horizons. The question we aim to answer is to what extent each
6 of those approaches relies to the long-term behaviour of the autoregressive rollout.
7 In other terms, we wish to find out what are the main drivers for the drift of a
8 trajectory, and how this differs for existing models. Overall, a formalization of
9 what is the drift and which component are important to reduce it is missing.

10 1 Introduction

11 What leads an autoregressive-model to drift away from a ground-truth trajectory ? What is the role
12 of the single-step prediction in the accumulation of error ? How do such dynamics depend on the
13 dataset in question ? How do different models accumulate different types of errors ? Those are the
14 questions we aim at answering.

15 2 Motivation : Why is that important ?

- 16 • First of all, to better validate the claimed properties of a new model. For instance, if a
17 model is claimed to better capture the high-frequencies, one should be able to show that the
18 long-term behaviour actually benefits from better captured high-frequencies.
- 19 • To identify the remaining challenges / parts to improve on a given (dataset, model pair): If
20 the model suffers from exposure bias (going out-of-distribution and performance suffering
21 from this), there should be a specific solution for this. If there is no exposure bias, the
22 one-step prediction and how it emphasizes some frequencies w.r.t others should be the most
23 important part. If it's a conjunction of both, how to address those?
- 24 • Currently, most works look at: MSE over the rollout / Time of high correlation with the
25 ground-truth, and then validate that the physical consistency is maintained by looking at
26 metrics such as TKE, evolution of the energy spectrum. The latter however don't give a
27 diagnosis on which aspects the model is better than the latter: is it short-term accuracy, no
28 exposure-bias, better captured frequencies ?
- 29 • To know what to expect from a model on a new dataset, as well as for reproducibility: by
30 looking only at the MSE of the model, a new practitioner/researcher doesn't know how a
31 model should behave/easier debugging

3 Background : ML-based Fluid Simulators

4 Sources of error amplification in autoregressive models

4.1 Definitions : One-Step error, Systematic-Drift and Exposure-Bias

We assume access to a neural-operator NO , which given only access to an initial condition x_0 , aims at predicting the subsequent states $\{\hat{x}_1, \dots, \hat{x}_K\}$ that closely match the ground-truth trajectory $\{x_1, \dots, x_K\}$, in an autoregressive manner :

$$\hat{x}_1 = NN(x_0) \quad (1)$$

$$\forall t \in \{2, \dots, K\} \quad \hat{x}_t = NN(\hat{x}_{t-1}) \quad (2)$$

Given the initial data distribution p_0 , we define the pushforward data distributions $p_t, t = 1, \dots, K$

$$X_t = GT(X_{t-1}), X_t \sim p_t, X_{t-1} \sim p_{t-1} \quad (3)$$

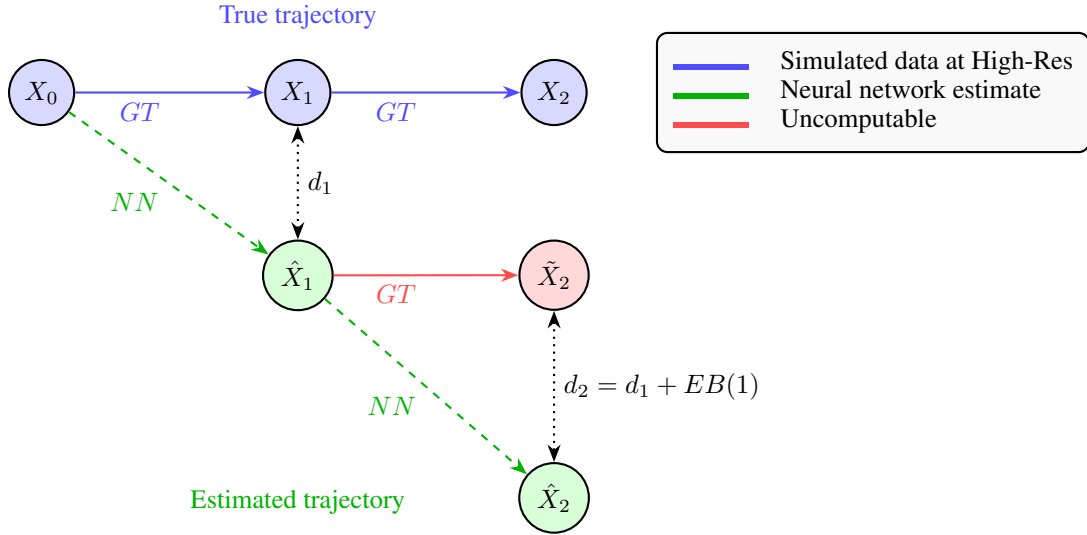
corresponding to the data distribution after t ground-truth simulation steps.

At a given timestep t , we write

$$\hat{X}_t = NN(\hat{X}_t) = X_t^{ID} + \epsilon_t^{OOD} \quad (4)$$

where X_t^{ID} is the sample in p_t that lies the closest (according to some metric to be defined) to $\hat{X}_t$.

We then decompose the error $\epsilon(t) = \|\hat{X}_t - X_t\| = \|NN^{(t)}\|$ as :



43

44 We have that :

$$\begin{aligned} \|\hat{X}_{t+1} - X_{t+1}\| &= \epsilon(t+1) \leq \|X_{t+1} - \tilde{X}_{t+1}\| + \|\tilde{X}_{t+1} - \hat{X}_{t+1}\| \\ &\leq \|X_{t+1} - \tilde{X}_{t+1}\| + \|GT(\hat{X}_t) - NN(\hat{X}_t^{ID})\| + \|NN(\hat{X}_t^{ID}) - NN(\hat{X}_t)\| \\ &= d_{TD}(t) + d_{OS}(t) + d_{EB}(t) \end{aligned}$$

45 Another possibility (11/03/2026):

$$\begin{aligned} \|\hat{X}_{t+1} - X_{t+1}\| &= \epsilon(t+1) \leq \|X_{t+1} - GT(\hat{X}_t^{ID})\| + \|GT(\hat{X}_t^{ID}) - NN(\hat{X}_t^{ID})\| + \|NN(\hat{X}_t^{ID}) - NN(\hat{X}_t)\| \\ &= d_{TD}(t) + d_{OS}(t) + d_{EB}(t) \end{aligned}$$

46 Another possibility - Without looking for in-distribution equivalent (11/03/2026):

$$\begin{aligned} \|\hat{X}_{t+1} - X_{t+1}\| &= \epsilon(t+1) \leq \|X_{t+1} - GT(\hat{X}_t^{ID})\| + \|GT(\hat{X}_t^{ID}) - NN(\hat{X}_t^{ID})\| + \|NN(\hat{X}_t^{ID}) - NN(\hat{X}_t)\| \\ &= d_{TD}(t) + d_{OS}(t) + d_{EB}(t) \end{aligned}$$

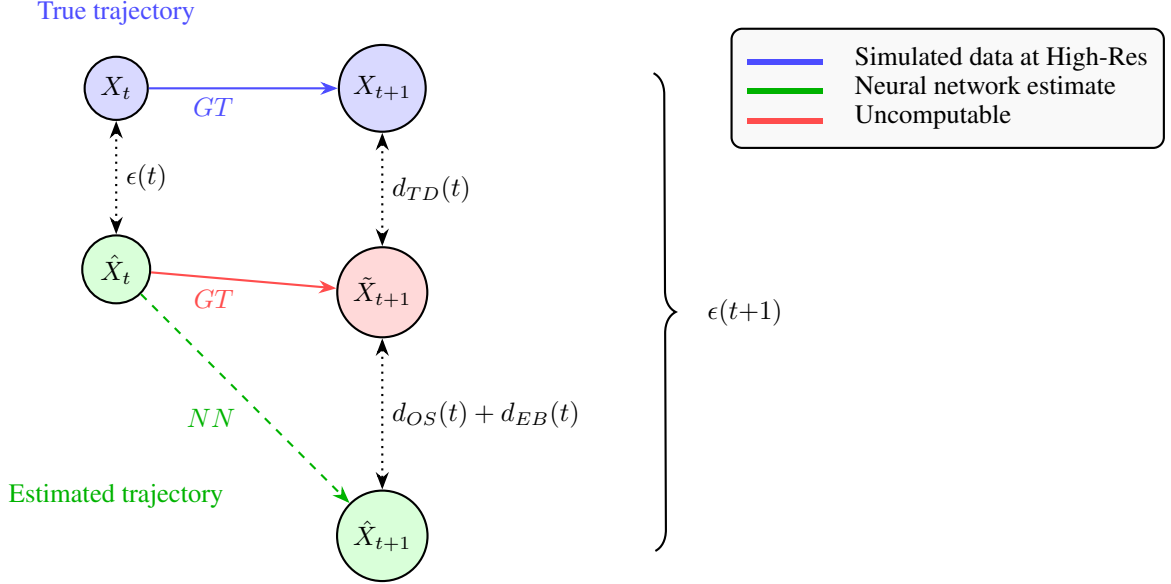


Figure 1: Decomposition into Systematic Drift, One-Step Error, and Exposure-Bias

47 where each term represents, respectively:

- The *Trajectory-Drift* error $d_{TD}(t)$ resulting from the accumulation of errors from earlier steps, and which only depends on emulator calls up to $t - 1$.
- The *One-Step* error $d_{OS}(t)$ which is the systematic error incurred at every step. If we assume that the ground-truth distribution stays constant during the simulation, i.e. $p_t = p_{t-1}$, we can write $d_{OS}(t) = d_{OS}$
- The *Exposure-Bias* error $d_{EB}(t)$ which is the additional error due to the discrepancy between $\hat{p}_t$ and p_t .

55 Note that after a single simulation step, since the input to the model is the in-distribution groundtruth
56 x_0 , the error only contains one-step error: $\|\hat{X}_1 - X_1\| = d_{OS}(0)$.

57 4.2 Identifying the contribution of each factor

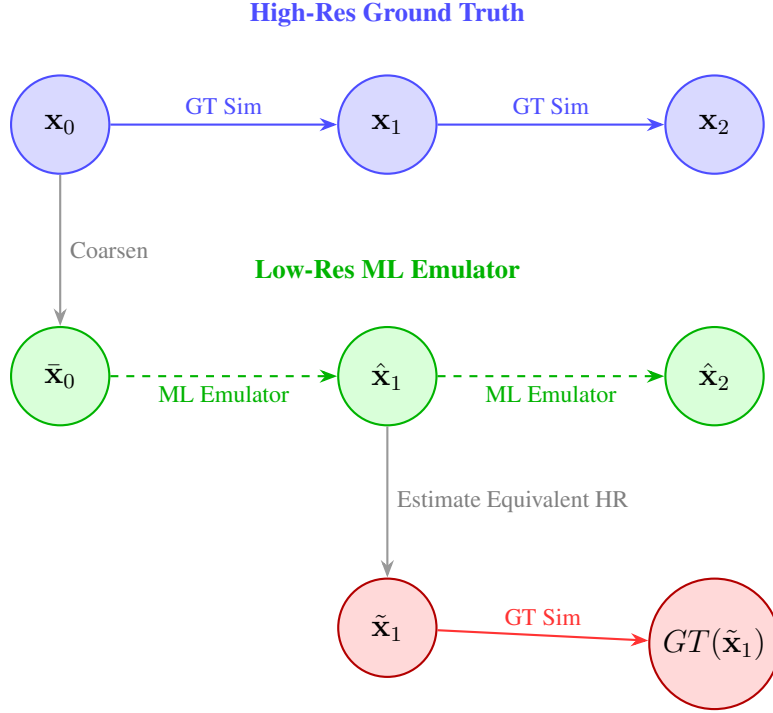
58 For a given model and dataset, we aim at giving a characterization of the sources of error in order to
59 design tailored improvements. However, there are two issues:

- In general one only has access to a high-resolution simulator GT^{high} , therefore evaluations of GT on arbitrary generated points $\hat{X}_{t+1}$ are inaccessible.
- It is not clear how to compute the in-distribution equivalent $\hat{X}_t^{ID}$

We argue that typically reported metrics, such as overall MSE over a trajectory or high-correlation time, don’t give enough information on the reasons why a model drifts, and are not satisfactory, especially for long rollouts where random effects can largely influence the generated trajectory.

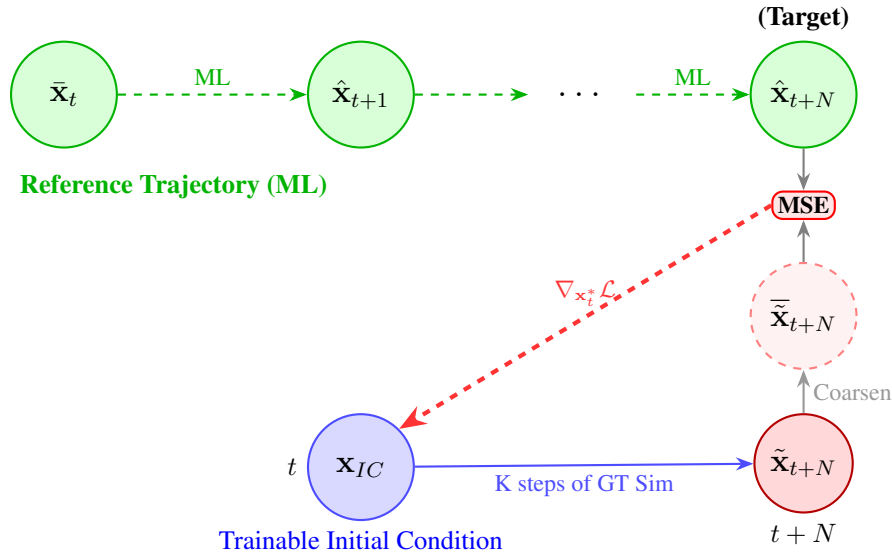
Furthermore, we argue that for each dataset, the main driver of drift w.r.t the ground-truth trajectory can be vastly different. We use this to motivate the fact that, models developed for a given task (such as PDERefiner for KolmogorovFlow and Kuramoto-Sivashinsky, which are inherently small one-step error dependent, are not suitable for datasets where the main error driver is exposure bias and systematic drift, such as Transonic Flow).

4.3 Obtaining a proxy for the model Ground-truth prediction



72

73 The estimate $\tilde{x}_1$ must be very precise, in order for the GT transition to be accurate. A super-resolution
 74 operation based only on $\hat{x}_1$ is intuitively not sufficient. Therefore, our approach consists in optimizing
 75 the initial condition of a chain of high-resolution simulations to make the final coarsened state match
 76 the low-resolution model prediction.



77

78 Using more GT Sim calls leads to a more physically-plausible state (it acts as a regularizer, i.e. after
 79 only a few simulator calls, the output may still contain non-physical artifacts), but it's also more
 80 expensive, and potentially harder to optimize. Also since the problem is under-determined (the target
 81 is a low-resolution field for which there may be infinitely-many relevant high-resolution fields). There
 82 regularization terms could also be explored to constrain the space of high-resolution states.

83 The main issue of this method is that it is slow: it runs backpropagation operations at high-resolution.
 84 It is furthermore required to run this optimization for every datapoint, and every autoregressive call.

85 It also requires a high amount of GPU RAM (already 30 GB for batch-size of 5 and 2 simulator
86 calls)... If one only uses this approach post-training as an analysis tool, this is probably fine.

87 As an alternative to retraining an initial condition for every (data index, AR step) pair, one could train
88 a neural network to map the initial condition to the refined one (this would be less memory expansive,
89 at the expense of being accurate). Note that in this problem we actually want to overfit, since we want
90 to exactly match the model's predicted state.

91 **5 The trajectory drift**

92 Assuming that we can write the

93 The trajectory drift is influenced by the "shape" of the error. In other words, starting from a
94 same initial condition X_t , two trajectories from two different neural operators $\hat{X}_{t+1}^{(1)}$ and $\hat{X}_{t+1}^{(2)}$
95 with a same one-step error ($d_{OS}^{(1)}(t) = d_{OS}^{(2)}(t)$), may have a different next-step trajectory drift :
96 $d_{TD}^{(1)}(t+1) \neq d_{TD}^{(2)}(t+1)$. This is because a given trajectory may diverge faster from the ground-
97 truth trajectory depending on which errors are made compared to that trajectory.

98 **5.1 The Pareto-front of frequency errors : Highlighting the trajectory drift effect**

99 Assuming that a neural emulator has a fixed one-step error, and no exposure bias. We aim at
100 identifying the response of the trajectory drift to the shape of the one-step error. Since we once again
101 don't have access to a ground-truth simulator to which one could inject an error of a given shape, we
102 make use of two neural emulators, and look at the linear combination between them.

103 We look at applying transformations to the two model's predictions

104 **6 Finding the optimal trade-off between Exposure-Bias and Performance**

105 As mentioned, it is not sufficient to look at training process error, as it could lead to exposure bias.
106 Furthermore, it is also not sufficient to look at the sampling process error, as the reduction of error in
107 a given timestep may come from reduction in other timesteps. Our aim is to isolate the effect that
108 a given timestep will incur in the next timesteps, as exposure bias. in previous timesteps directly
109 impact error in current timestep.

110 We highlight a few points:

- 111 • if a model has exposure bias then it is likely that
- 112 • How close the input error needs to be
- 113 • How much of the "exposure-bias" comes from the fact that the model is not supposed to be
114 deterministic ? Rather than drifting ?

115 **6.0.1 Model Cross-Comparisons : A proxy metric for exposure bias**

116 The intuition is the following: Given the trajectory $\{\hat{x}_1, \dots, \hat{x}_K\}$ obtained from autoregressive sampling
117 of NO_1 , if both models have a low exposure bias, then the average distance between $\hat{x}_t$ and
118 $NO_2(\hat{x}_{t-1})$ should not vary much as a function of t . Note that this is only a sufficient condition:
119 if the average distance between NO_1 and NO_2 stays constant, it doesn't imply that they are both
120 exposure-bias free.

121 By using multiple "reference" (evaluation) models, we can (potentially?) get rid of the case where
122 the distance between models doesn't increase with constant distance. We may also be able to obtain a
123 more accurate estimate of each model's biases.

124 In particular, we look at the evolution of:

$$\mathbb{E}\|\hat{x}_t - NO_2(\hat{x}_{t-1})\|^2 = \mathbb{E}[\epsilon(x_t)^2 + \|NO_2(\hat{x}_{t-1}) - GT(\hat{x}_{t-1})\|^2] \quad (5)$$

$$- 2\langle \hat{x}_t - GT(\hat{x}_{t-1}), NO_2(\hat{x}_{t-1}) - GT(\hat{x}_{t-1}) \rangle \quad (6)$$

$$= \mathbb{E}\left[(\epsilon_{NN_1}(x_0) + EB_{NN_1}(\hat{x}_{t-1}))^2\right] \quad (7)$$

The second term is the single-step error made by the second neural-operator. It can also be decomposed as a sum of the systematic drift of NO_2 , and the exposure-bias of NO_2 when using the trajectory of NO_1 as input.

In practice, we observe that a cross-correlation doesn't necessarily induce a faster increase of error. The reason could be two things:

- The cross-correlation is not a good indicator of the exposure bias ? Indeed, the cross-correlation implies that the samples generated by the neural emulator become out-of-distribution. It however doesn't necessarily imply that the model suffers from this drift. Still, is it possible that two models get further from each other and their performance does not degrade ? Can we think about 2-steps metric to show that a model indeed suffers from its own drift ? Like $Eval(M1(M1(x)))$ vs $M1(M1(x))$ (regarde carnet).
- If use one model to evaluate, could just mean that it is more sensible to a certain type of drift than another. Can either : evaluate with more models. Actually, the cross-comparison of a diffusion model with a certain schedule and a diffusion model with another schedule is non-increasing (it even decreases).
- The cross-correlation is a good indicator of the exposure bias, but exposure bias becomes insignificant in the error accumulation (due for instance, to dynamics). In particular, at the time where exposure bias is too high, the trajectory has maybe drifted so much that the error cannot be seen.
- How to explain that evaluating a diffusion model traj. with a diffusion model evaluator leads to a reduction of error as a function of the simulation step ?
- The evaluator is the one that suffers from the out-of-distribution : the discrepancy at a given timestep could hinder the evaluator rather than the model itself.

6.1 Ideas of cross-comparisons evaluations

- On the trajectory of M_1 , apply M_2 (with less exposure bias, for instance a 2-steps Unet which has no out-of-distribution samples). We assume that this will "put back" the sample more in-distribution. Then apply M_1 again, and compare to applying M_1 twice.

6.2 The exposure bias of different neural emulators

7 Experiment log

7.1 Phases of the project

PHASE 1: Identify how HR proxy emulation works - Are there any troubles with it - What are the metrics of interest to measure how good the estimate is - What is the correct way to initialize the initial condition (before optimization) : - initial condition may influence how easy it is to obtain a correct or incorrect solution to the optimization problem - Explore how the dataset influences the correctness of the estimate, w.r.t the defined metrics - Effect of the hyperparameters (number of proxy simulator calls, number of optimization steps, type of input at initialization, regularization of the prediction)

There, the main point is ensuring the unconditional (dataset and model independent) correctness of the proxy estimator. If the proxy is incorrect, the downstream uses of it are useless

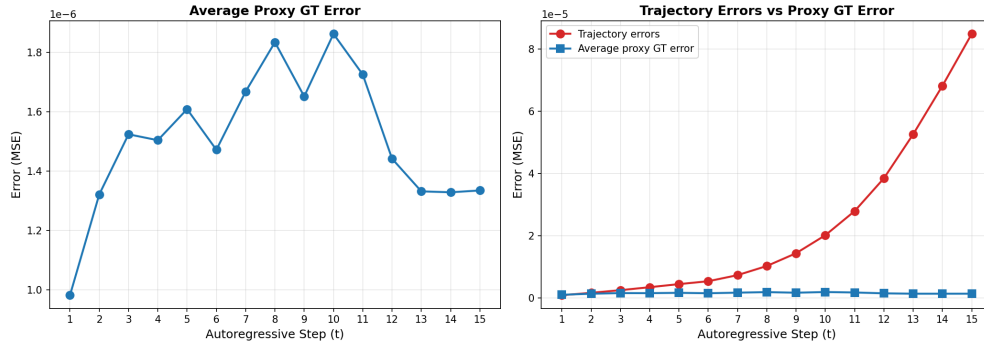
PHASE 2: Exploring the potential downstream applications of the HR proxy emulation

7.2 Logs

13.02.2026 An important issue seems to be the computational overhead required to optimize the initial condition in high-resolution. For a batch size of 4, and 15 AR steps, 1000 steps of optimization take more than 10 hours. The question is whether we can juggle with the parameters (increasing model calls and decreasing number of steps) to cope with that. The issue is that we will anyway not be able to run the experiment on a batch of 64, let alone the full validation set.

171 → Attempt to cope with that : look at effect of number of optimization steps on accuracy at ts.1.
 172 Explore the NB STEPS x MODEL CALLS GRID
 173 Result : for Kolmo, 5 calls seems to be a good starting point. Also, lower learning rate (0.001) leads
 174 to less explosion.

14.02.2026 Obtaining results on a single trajectory, for Diffusion Adaptive UT :



175

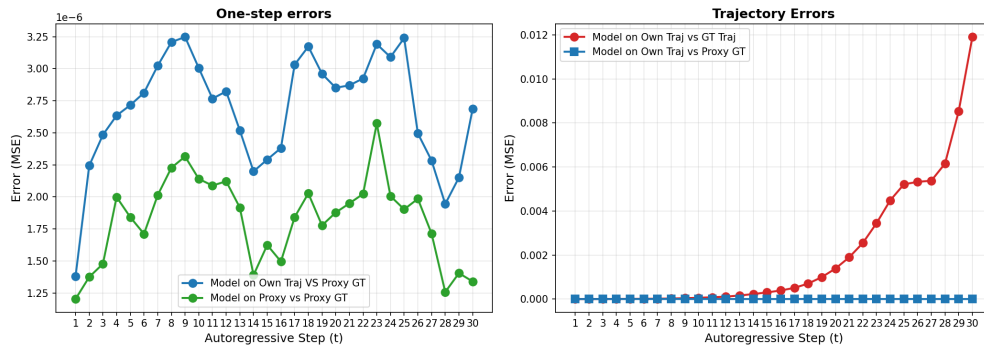
176 It's not sure whether the increase is due to :

- 177 • (1) Harder optimization at later steps
- 178 • (2) Harder One-step error depending on position on trajectory
- 179 • (3) Actual Exposure-Bias

180 (1) The type of initialization "model-traj" is good for steps larger than n simulator calls, but for earlier
 181 steps it still uses the GT traj, which could lead to unequal treatment. (2) Adding the Proxy GT vs
 182 Model on Proxy GT Error as a proxy should get rid of point. This should give a closer baseline, if the
 183 one-step error fluctuates based on the position in the trajectory.

184 Overall, a guarantee of the correctness of the Proxy is very important.

15.02.2026 Now with the Proxy GT vs Model on Proxy GT:



185

186 The two curves follow each other closely, showing the fluctuations are due to the fluctuations in the
 187 trajectory rather than in exposure-bias / estimate correctness. This at least shows that the proxy is
 188 able to consistently (in the same way) match the model trajectory (for timesteps ≥ 2).

189 We see an increase in the ratio between the two curves at timestep 2, and then close to no increase (at
 190 eye view).

191 The question is whether this increase is due to exposure-bias, or due to bad calibration/convergence
 192 of the proxy. Also, could just be that the distance is due to the fact that there is no high-resolution
 193 proxy which can match the low-resolution model prediction.

194 **Might not be able to give a certificate, but at least can we put the initial prediction in "disad-**
195 **vantage", i.e. make initial condition further away from trajectory**

196 **16.02.2026** Meeting with Felix. Advices: start with 1D cases where we don't actually need to run
197 a coarsening operation. Simple cases would also allow to have a better characterization of what it
198 means to be "in-distribution" (Fourier approach). Furthermore, one can iterate faster (his advice is :
199 simulation should take in order of seconds, training should take in order of 10 minutes).

200 This makes sure that we have full vision (for instance, in the HR case, we only compare the LR fields,
201 so the HR potentially could be out-of-dist). Multi-resolution could be a next step.

202 So start with advection, KS even, Burgers. Or chaotic case where we can validate that drifting obeys
203 to chaos theory.

204 The plan is therefore to start with exponax, compute the initial condition for a lot of trajectories, get
205 the exposure-bias. Maybe we can also get a characterization of the error separation.

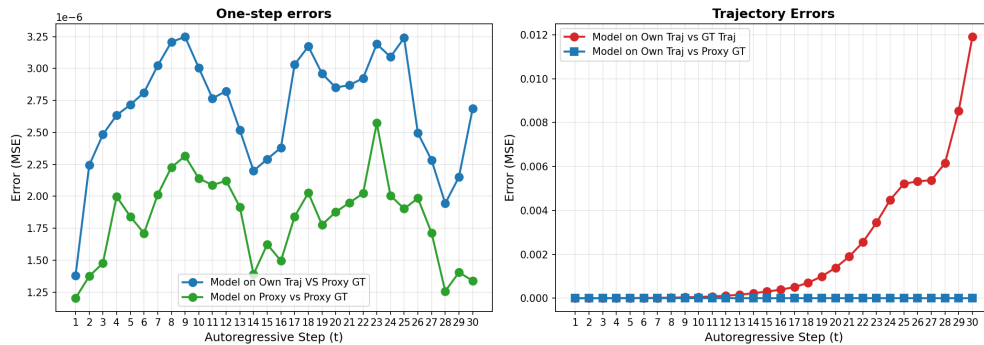
206 **18.02.2026** First runs on APEBench, 1D Advection, checking the different training options (one,
207 sup unrolling, diverted chain unrolling), with advection gamma equal to 10.5

208 On this case, multiple questions: - one has a better first-step mse than sup;2 and sup;5, which is
209 intuitive - however, one has a worse first-step mse than div;2 and div;5, which is not intuitive (check
210 myself what div means, and ask Felix if doesn't make sense still) - we also see that among one, sup;2
211 and sup;5, sup;5 has worse first-step MSE, but best long-term MSE, which is intuitive. Question is
212 whether this difference is due to exposure-bias reduction, or to predictive placement.

213 I think that, without diving into Exposure-Bias etc, the first hypothesis is that first-step MSE does
214 a big part of the job. This sounds obvious, but this is probably a finding to display. In the sense
215 that now, when creating a new architecture/model, one should display which part of the trajectory
216 improves. Check which proportion of the PDEs is defined by the one-step MSE,

217 There is also the hypothesis made in APEBench that unrolling could increasing the receptive field
218 and therefore lead to better one-step prediction ? Not sure though. It's also claimed that the diverted
219 training is a good trade-off between one-step MSE and long-term performance. Indeed, that's the
220 case. Using that method is therefore a matter of what's important for a given design in mind.

221 Ran experiments on FNO:



222 My hypothesis is that the only thing that architectures modify is the one-step MSE, in most cases.
223 Especially, when training them in one-step, architectures in general have the same error growth.
224 Actually, that's not true for all architectures (and a regularization method can be thought of as being
225 part of the architecture, for instance adding noise on input in diffusion models as in ACDM), but in
226 most cases it is true. The question is rather, can we categorize architectures into categories of brought
227 improvements. for instance that training with noiThey don't bring an

228 Idea for the diffusion project: What about using different models, maybe also during inference. Could
229 also vary the architectures, if it turns out that an architecture is better for a given noise range. This
230 would increase the model size by 3 or 4, but in a smart way. And if we can show it improves the
231 performance better than a classical ensemble, would be nice.

232 But for now, let’s say we want a single model: we can train a model fully on the lowest noise level,
 233 actually just pick the one with the lowest sampling error (rather than taking a parameter τ). Then
 234 we train earlier step, using binary search. And iterate. Or we train it all at the same time, and

235 8 Discussions with Claude

236 8.1 Frequency distribution of one-step error and its effect on rollout

237 A natural question is whether two models with the same total one-step MSE but different spectral
 238 error profiles will exhibit different rollout behaviour. By Parseval’s theorem, the MSE decomposes
 239 exactly into per-mode contributions:

$$\|\hat{x} - x\|^2 = \sum_k |\hat{x}_k - \tilde{x}_k|^2 \quad (8)$$

240 so at the single-step level, MSE is frequency-agnostic: an error of magnitude ϵ at wavenumber k_1
 241 contributes identically to an error of the same magnitude at wavenumber k_2 .

242 However, the picture changes drastically when we consider autoregressive rollout. The key observa-
 243 tion is that the *trajectory drift* $d_{TD}(t)$ depends not only on the magnitude of the one-step error, but
 244 on its spectral distribution.

245 **The general case: forward energy cascade.** In most 3D turbulent systems, energy cascades from
 246 large scales to small scales (forward cascade). In such systems:

- 247 • **Low-frequency errors** corrupt the large-scale structure of the flow. Since the large scales
 248 govern advection and the overall trajectory of the system, these errors propagate to all scales
 249 through nonlinear mode coupling. A wrong large-scale flow transports features to wrong
 250 locations, leading to rapid trajectory divergence.
- 251 • **High-frequency errors** are noise-like and tend to be damped by physical dissipation
 252 (viscosity). They remain spectrally localized for short times and have a weaker feedback
 253 onto the large-scale dynamics.

254 Therefore, a model whose one-step error is concentrated in low frequencies will typically exhibit
 255 faster trajectory drift than one whose error is concentrated in high frequencies, even at equal total
 256 MSE. This favours architectures like UNets, whose spatial locality and pooling hierarchy naturally
 257 capture low-frequency structure well, even if they smear high-frequency detail.

258 **The inverse cascade case: Kolmogorov flow.** Kolmogorov flow is 2D Navier-Stokes with sinu-
 259 soidal forcing at a fixed wavenumber, and is one of the most common benchmarks in the neural
 260 operator literature. Crucially, 2D turbulence exhibits an *inverse energy cascade*: energy injected at
 261 small scales flows upward to larger scales, while enstrophy cascades downward.

262 In this regime, the argument reverses. High-frequency errors can feed energy upward into the large-
 263 scale structure through the inverse cascade, eventually corrupting it. Getting the small scales right
 264 therefore matters for the large-scale trajectory, which means the FNO’s purported advantage at high
 265 frequencies could genuinely translate into better rollout stability on this specific benchmark.

266 Other systems exhibiting inverse cascades include rotating and stratified (geophysical) flows, thin
 267 fluid films, and certain MHD configurations.

268 **Role of the Reynolds number.** The Reynolds number $Re = UL/\nu$ controls the ratio of inertial to
 269 viscous forces and determines the width of the inertial range over which the cascade operates.

- 270 • At **low Re** (typical of many benchmarks, $Re \sim 100\text{--}1000$), viscosity damps fluctuations
 271 efficiently across scales. The inverse cascade has limited room to operate, and errors at small
 272 scales are dissipated before they can migrate significantly upward. The spectral distribution
 273 of error has a moderate effect on rollout.
- 274 • At **high Re** ($Re \gtrsim 10000$), the inertial range is wide and the inverse cascade is vigorous.
 275 High-frequency errors have many scales to climb through and can seriously corrupt large-
 276 scale structures during rollout.

277 This means that model comparisons are Reynolds-number dependent: a UNet and an FNO may
 278 show similar rollout behaviour at moderate Re, while at high Re the difference in their spectral
 279 error profiles becomes decisive. This should be taken into account when designing benchmarks and
 280 interpreting results: the Re at which a benchmark is run determines how much the spectral structure
 281 of the one-step error matters for long-term trajectory fidelity.

282 **Implications for benchmarking.** This analysis suggests that reporting only aggregate one-step
 283 MSE (or even rollout MSE) is insufficient. One should additionally report:

- 284 • The spectral decomposition of the one-step error (per-wavenumber MSE)
- 285 • The Reynolds number and cascade regime of the benchmark system
- 286 • Whether rollout improvements stem from better low-frequency or high-frequency accuracy

287 This would allow distinguishing whether a claimed architectural improvement (e.g., “better high-
 288 frequency capture”) actually benefits the downstream rollout task, or whether it merely redistributes
 289 error across frequencies without affecting trajectory drift.

290 **A 1D test case: Kuramoto-Sivashinsky.** The frequency-dependent rollout effect can also be
 291 observed in 1D, using the Kuramoto-Sivashinsky (KS) equation. The KS equation features a linear
 292 instability band at intermediate wavenumbers that injects energy, while the quadratic nonlinearity
 293 $u \cdot u_x$ redistributes energy across scales. Crucially, this redistribution goes *both directions*: forward to
 294 high wavenumbers (where the hyperdiffusion u_{xxxx} dissipates it) and backward to low wavenumbers.
 295 This partial inverse transfer means that high-frequency errors can feed into low- k modes through
 296 nonlinear interactions.

297 The linear growth rate per wavenumber k is:

$$\sigma(k) = \underbrace{|\gamma_2| k^2}_{\text{anti-diffusion (energy injection)}} - \underbrace{|\gamma_4| k^4}_{\text{hyper-diffusion (dissipation)}} \quad (9)$$

298 Modes are linearly unstable where $\sigma(k) > 0$, i.e. for $k < k_c$ where:

$$k_c = \sqrt{\frac{|\gamma_2|}{|\gamma_4|}} \quad (10)$$

299 With default parameters ($\gamma_2 = -1.2$, $\gamma_4 = -15.0$), this gives $k_c \approx 0.28$. To widen the instability
 300 band so that the task relies more on high-frequency accuracy, one increases the ratio $|\gamma_2|/|\gamma_4|$, pushing
 301 k_c to higher wavenumbers.

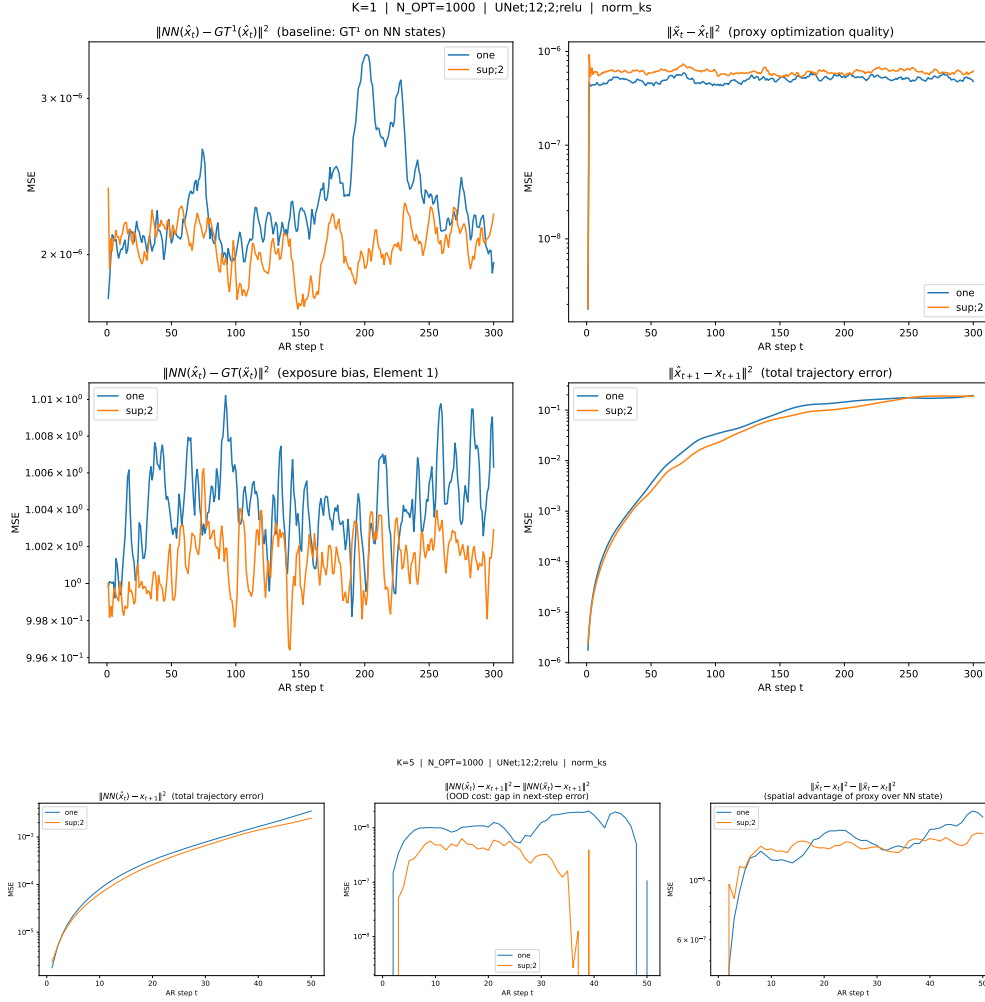
302 Additionally, the KS equation is chaotic (positive Lyapunov exponents), so any perturbation grows
 303 exponentially. However, the rate of growth depends on which modes are perturbed: errors in the
 304 linearly unstable band (the most dynamically active wavenumbers) are amplified fastest.

305 This makes KS a suitable 1D testbed for the spectral error experiment: given two models with the
 306 same total one-step MSE, one with error concentrated at low- k and one at high- k , we expect different
 307 rollout divergence rates. The chaotic dynamics should amplify the difference, and the partial inverse
 308 transfer should make high-frequency errors more consequential than in a purely forward-cascade
 309 system.

310 By contrast, Burgers’ equation is less informative for this analysis: it is not chaotic (shocks are
 311 attracting structures), so errors are either absorbed into shocks or advected away rather than cascading
 312 across scales.

313 **10.03.2026** The issue with the approach is that the proxy GT is not able to match well the ML
 314 simulator trajectory, and the error is much lower for $t = 0$ than for even $t = 1$. For $t > 1$, it appears
 315 the error between the proxy GT and the ML simulation is stable for unrolled steps. See top right plot,
 316 which shows error between ML trajectory, and proxy step, for $K = 1$. Behaviour is similar for larger
 317 K .

318 Actually, let’s try to not compare the model’s prediction with the GT on the GT proxy, as it may be
 319 arbitrary far away. What we want to know is whether a ML model suffers from its own distribution

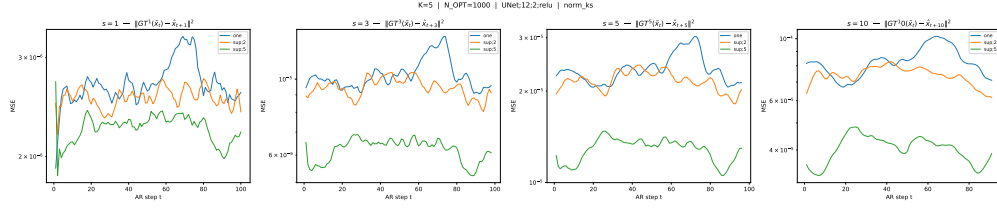


more than it would suffer from a GT state. By suffering, we therefore mean that its error is further away from the GT trajectory (total trajectory error). Therefore, at each step in the ML trajectory, we can run the ML model on the proxy GT, the ML model on itself, and compute the 2 errors w.r.t the GT trajectory. Also one must compute the difference in error between $MSE(NN(x_{hat}), x_{gt})$ and $MSE(NN(x_{tildex}, x_{gt}))$ to ensure that the cost of out-of-dist input is not just due to the proxy gt being closer to the true trajectory.

10.03.2026 - 2 In previous figure, it appears there is a difference in error between calling NN on its own traj, vs calling it on the GT traj, however, there is also a difference in the original error (spatial advantage). I don't think it's a relevant metric to compare to the true GT because it quickly accumulates too much.

An alternative is: at a given step t , compute the GT proxy. Then for a given s , run s GT steps from there, and look at the error between $\hat{x}_{t+k}$ and $GT^k(\tilde{x})$. Look at how this error evolves with time. We expect that a model suffers from its out-of-distribution if the error increases. Since the proxy estimate's error, as seen above, doesn't increase as t increases, it's likely that the metric is accurate.

10.03.2026 - 3 We need more than just a trend. We need a precise characterization of each term. We want to know: at a given timestep, what is the contribution of each term, as in Figure 4.1. First, we need to check if the sum of the three terms actually reaches the full trajectory error at a given step. This would be a first point to know if the characterization can be done.



338 **11.03.2026** In order for the decomposition to be useful, we need to either show that it's exact, or at
 339 least show that it's able to rank different models in the same order as they are ranked in terms of the
 340 trajectory drift, at each timestep. If the models are not ranked similarly, then the metric isn't useful.

341 After running experiment, it seems that there is full-agreement (the trend is good), but there is
 342 always an under-estimation of the error, especially at the beginning. We need to check whether the
 343 specification of each term makes sense, whether there is a given term underfitting, or whether the
 344 sum of squared terms is just underestimating too much.